Criticality-Aware Object Placement Across Tiered Memory in OCaml 5

Andrew Li
Northwestern University
Evanston, IL, USA
andrewli@u.northwestern.edu

1 Background

Data intensive workloads consume large quantities of DRAM, which is increasingly expensive given the growth of AI data centers. This memory bottleneck motivates a transition to heterogeneous memory hierarchies, where local DRAM is augmented by cheaper "far memory" tiers like Compute Express Link (CXL)-attached memory [2]. Traditional OS-level paging lacks insight into application object structures, leading to suboptimal placement of memory into pages across memory tiers. Heuristically, it may seem reasonable to place objects based on hotness or access frequency [6], but modern research has shifted towards criticality-based paging, which uses not just hotness as a heuristic but also memory-level parallelism (MLP). The intuition behind criticality-based policies is that some data structures may be more friendly towards hardware-level parallel accesses, while other data structure with heavy pointer chasing are not. The OS has no understanding of these data structures, only language runtimes do. Managed runtimes with garbage collection (GC) leverage OS-level paging mechanisms to manage large heaps of data structures with varying degrees of MLP; this object mobility is a fundamental advantage of managed over unmanaged languages [5]. Optimal paging of these data structures is critical for applications written in high-level programming languages such as Python, Julia, Haskell, and OCaml. Frameworks at the OS-level, such as PACT with a Per-page Access Criticality metric [1], are insufficient for runtimes themselves.

2 Project

The project is to introduce compiler support for runtime-level criticality metrics that enable placement of data structures on heaps spanning various tiers of memory. This project requires modification of both the garbage collector and compiler for a production language. We choose to use OCaml 5 [3] because its concurrency offers a interesting level of multi-core complexity. OCaml's garbage collector is inherently tiered which makes it intuitively well-suited for research on tiered memory. A drawback of using OCaml is that its compiler does not use LLVM as a backend, which makes an attribute-based modification harder to robustly compare with other languages. We leave this as future work.

At the garbage collector level, we modify the OCaml runtime to support both main-memory major heap pages and far-memory major heap pages. The design of the OCaml garbage collector is with a stop-the-world (STW) minor heap that is, by default, 2 MB in size. Once the minor heap fills, all live objects are promoted to the major heap. The first half of the project is to re-engineer the promotion step to support both promotion to a main-memory major heap page it or a far-memory major heap page. The second half of the project is at the compiler level, where we introduce attributes like `[@far_memory]` to support transparent paging, which contrasts from similar compiler work like TrackFM [4] that relies on unmodified code. The difference is that, lacking annotations, TrackFM must be more middle-end, whereas the introduction of attributes enables a more front-end level contribution.

3 Implementation Plan & Timeline

The project is naturally split into two halves. The first half is runtime, garbage collection, and memory management. At a high level, the specific aims are

(1) Modify the runtime to support multiple tiers of major-heap pages spanning both DRAM and CXL-based far memory. We can simulate far memory as a NUMA node with an added Gaussian latency.
(2) *Identify* criticality metrics that can be used at a runtime level, taking inspiration from prior work such as PACT [1].
(3) Implement these criticality metrics at the runtime level, incorporating both access frequency (hotness) and MLP.
(4) Re-engineer the minor heap promotion step to make placement decisions and promote live objects to either a main-memory or far-memory major heap page based on criticality metrics.

The second half is OCaml compiler front-end modifications.

(1) Introduce source-level attributes (e.g., [@far_memory]) in the OCaml frontend that annotate data structure allocations with placement hints.
(2) Thread these attributes through the compiler pipeline such that they are visible at allocation sites in the runtime. The most likely implementation of this is to somehow embed source-level attributes in the metadata bits of each object. If there are insufficient bits for this, other means must be explored.
(3) Ensure the GC respects these source-level attributes when making placement decisions.

3.1 Timeline

Week 4 (4/20-4/26) — Runtime scaffolding Modify the OCaml runtime to support two major-heap arenas: one backed by local DRAM and one by a NUMA node with injected Gaussian latency simulating CXL. Spawn

a VM with NUMA nodes. Use `mmap` to create the two separate arenas. Verify allocation and GC correctness by writing and running some basic programs, as well as some memory intensive programs, and maybe by force triggering the GC. The author's code from Advent of Code 2025 may contain useful programs to run.[1]

Week 5 (4/27-5/3) — Promotion path Re-engineer the minor heap promotion step to route live objects to either arena. Use a trivial policy (e.g., round-robin) initially to confirm end-to-end plumbing.

Week 6 (5/4-5/10) — Criticality metrics Identify and implement runtime-level PAC-inspired metrics: per-object hotness counters and an MLP proxy (e.g., outstanding load depth at GC time). Wire metrics into the promotion decision.

Week 7 (5/11-5/17) — Compiler attributes Add `[@far_memory]` and `[@main_memory]` as recognised attributes in the OCaml parser and typechecker. Propagate it through the lambda and Cmm IRs and embed it in the object header or a side table visible at allocation sites.

Week 8 (5/18-5/24) — GC integration Make the GC read the embedded attribute and override or bias the heuristic placement decision. Validate correctness on small hand-written tests with known MLP profiles.

Week 9 (5/25-5/31) — Evaluation Write and run benchmarks: pointer-chasing (low MLP) and array-traversal (high MLP) workloads under memory pressure. Collect results for all four conditions (all-main, all-far, heuristic, attributed). Make figures.

Week 10 (6/1-6/9) — Buffer Polish benchmarks based on preliminary results. Since the author will be starting their internship around this time, talk to devs in the trading industry to get a sense of what they care about, and if time permits, implement it.

4 Evaluation

The project will be evaluated by evaluating the attribute-based approach against the runtime-only heuristic baseline, measuring performance on data-intensive OCaml workloads. Specifically, we will write workloads that use both low and high MLP data structures under high memory pressure. We can achieve memory pressure by lowering the size of the heaps. A baseline is to run the workload on high DRAM machines without memory pressure, which is the upper bound of performance. The negative control is to have every page promoted to far memory, which will have worse performance than the baseline due to the access latencies of the CXL-based (albeit emulated) memory. Our experiments are thus to

(1) Measure the effect on performance by using heuristics to place objects across tiered memory pages. The hypothesis is that heuristics are sufficient to increase performance relative to the all-far control.
(2) Measure the effect on performance by using compiler frontend attributes inserted by the programmer. The hypothesis is that these hand-written attributes increase performance more compared to the heuristics-only approach, and can achieve performance close to the all-main baseline.

[1] https://github.com/andrlime/aoc2025

5 Contributions

Our contributions are

(1) Compiler frontend patch, and garbage collector patch that enables both automatic, heuristics-based and manual, attributes-based placement of objects on heap pages spanning tiered memory.
(2) Custom-written benchmarks based on existing benchmarks from similar work, ported to OCaml, and both with and without attributes.

6 Team

Andrew Li The author is interested in networking and distributed systems, compilers, and high performance scientific computing. The author will be interning at a trading company starting May 26. The author therefore will need to present the final presentation over Zoom.

References

[1] Hamid Hadian, Jinshu Liu, Hanchen Xu, Hansen Idden, and Huaicheng Li. 2026. PACT: A Criticality-First Design for Tiered Memory. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*. ACM, Pittsburgh, PA, USA. doi:10.1145/3779212.3790198

[2] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *Comput. Surveys* 56, 11, Article 290 (July 2024), 37 pages. doi:10.1145/3669900

[3] KC Sivaramakrishnan, Stephen Dolan, Leo White, Sadiq Jaffer, Tom Kelly, Anmol Sahoo, Sudha Parimala, Atul Dhiman, and Anil Madhavapeddy. 2020. Retrofitting Parallelism onto OCaml. *Proceedings of the ACM on Programming Languages* 4, ICFP, Article 113 (Aug. 2020), 30 pages. doi:10.1145/3408995

[4] Brian R. Tauro, Brian Suchy, Simone Campanoni, Peter Dinda, and Kyle C. Hale. 2024. TrackFM: Far-out Compiler Support for a Far Memory World. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '24)*. ACM, La Jolla, CA, USA. doi:10.1145/3617232.3624856

[5] Nick Wanninger, Tommy McMichen, Simone Campanoni, and Peter Dinda. 2024. Getting a Handle on Unmanaged Memory. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*. ACM, La Jolla, CA, USA. doi:10.1145/3620666.3651326

[6] Zhen Xie, Jie Liu, Jiajia Li, and Dong Li. 2023. Merchandiser: Data Placement on Heterogeneous Memory for Task-Parallel HPC Applications with Load-Balance Awareness. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '23)*. ACM, Montreal, QC, Canada. doi:10.1145/3572848.3577497

A Gemini Deep Research

`https://gemini.google.com/share/f3387fef346c`

B AI Usage Policy

This project will be a yes-AI project. It is anticipiated that the benchmarks will not be written using AI.